

CS 4032 – Web Programming



MEAN STACK



Mongo DB
(database system)



Express
(back-end web
framework)

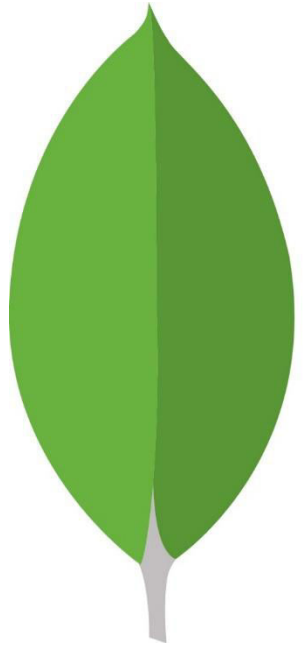


Angular.js
(front-end
framework)



Node.js
(back-end runtime
environment)

Collection of JavaScript based technologies used to develop web applications.



mongoDB

What is NoSQL ?

- Not SQL, Not Only SQL, *Not Relational*
- A term that describes a class of data storage and manipulation technologies and products that do not follow the RDBMS principles and focus on large datasets, performance, scalability, and agility

Types of NoSQL Databases

- Key-Value Stores
 - Used for large amounts of data with simple lookup queries
 - Example DB's are : Redis and DynamoDB
- Column Family Stores
 - large amounts of data with predictable query patterns
 - Example DB's are : Cassandra and HBase
- Graph Databases:
 - analyzing and traversing relationships between connected data
 - Example DB's are : Neo4j and Amazon Neptune
- Document Databases -
 - A *document* in a document database consists of a loosely structured set of key-value pairs.
 - Example DB's are : MongoDB and CouchDB

A "Document" Example

```
{  
  "first_name": "John",  
  "last_name": "Doe",  
  "age": 20,  
  "address": {  
    "street": "123 Main"  
    "city": "Los Angeles"  
    "state": "CA"  
  }  
}
```

- It's basically JSON
- Why is it called a *document*, not a *object*??

MongoDB

- The most popular NoSQL database
 - Document database
 - [BSON](#) data types
- The "M" in MEAN/MERN stack



Type	Number	Alias	Notes
Double	1	"double"	
String	2	"string"	
Object	3	"object"	
Array	4	"array"	
Binary data	5	"binData"	
Undefined	6	"undefined"	Deprecated.
ObjectId	7	"objectId"	
Boolean	8	"bool"	
Date	9	"date"	
Null	10	"null"	
Regular Expression	11	"regex"	

MongoDB

According to Wikipedia:

“ **MongoDB** (from *humongous*) is a cross-platform [document-oriented database](#). Classified as a [NoSQL](#) database, MongoDB eschews the traditional table-based [relational database](#) structure in favor of JSON-like documents with dynamic schemas (MongoDB calls the format [BSON](#)), making the integration of data in certain types of applications easier and faster. Released under a combination of the [GNU Affero General Public License](#) and the [Apache License](#), MongoDB is [free and open-source software](#).

– Wikipedia

Mongodb-Data

MongoDB stores data in a binary JSON-like format called BSON

- BSON supports boolean, integer, float, date, string and binary types
- MongoDB can also support large binary objects, eg. images and videos
- These are stored in chunks that can be streamed back to the client for efficient delivery

Mongo Db is Useful

- When data that is being generated by internet applications and IoT devices are non-structured which cannot be saved in a traditional SQL database.
- Mobile apps storing log data and user activity in unstructured format
- Many companies will store data before they know how it will be used later

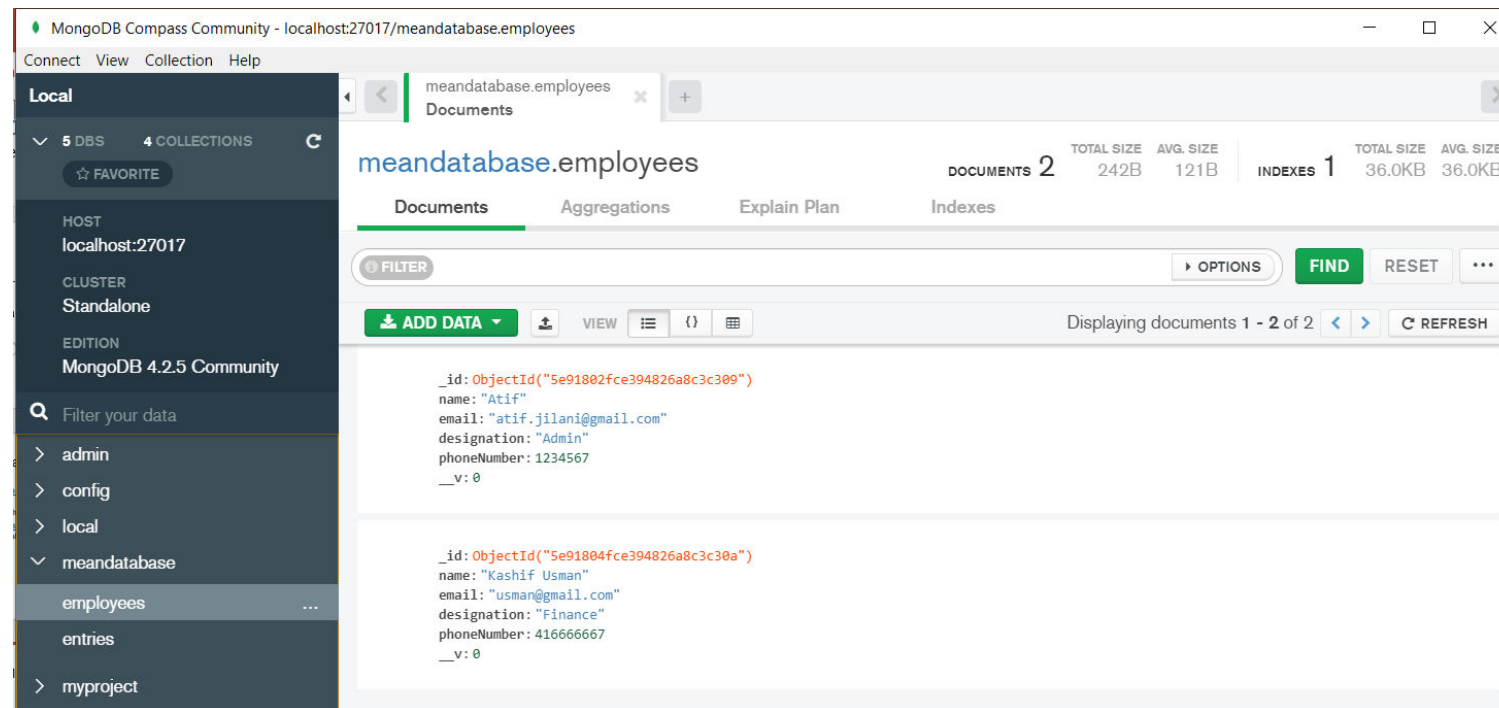
MongoDB Installation

- Install [MongoDB Community Edition](#)
 - No security by default
 - Localhost binding (default since 3.6)
- Install [MongoDB Compass](#)
 - MongoDB Windows Installer installs Compass by default

Basic Usage of MongoDB Compass

- Connection
- Create databases and collections
- Document CRUD

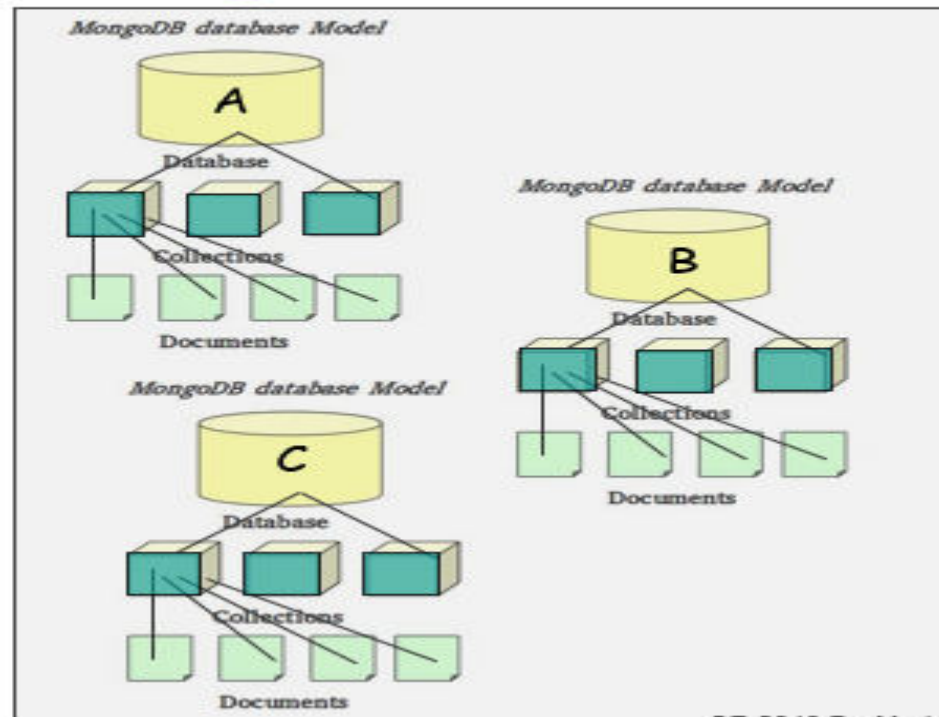
The GUI for MongoDB. Visually explore your data. Run ad hoc queries in seconds. Interact with your data with full CRUD functionality. View and optimize your query performance. Available on Linux, Mac, or Windows. Compass empowers you to make smarter decisions about indexing, document validation, and more.



Mongo Database Server



Mongo database server

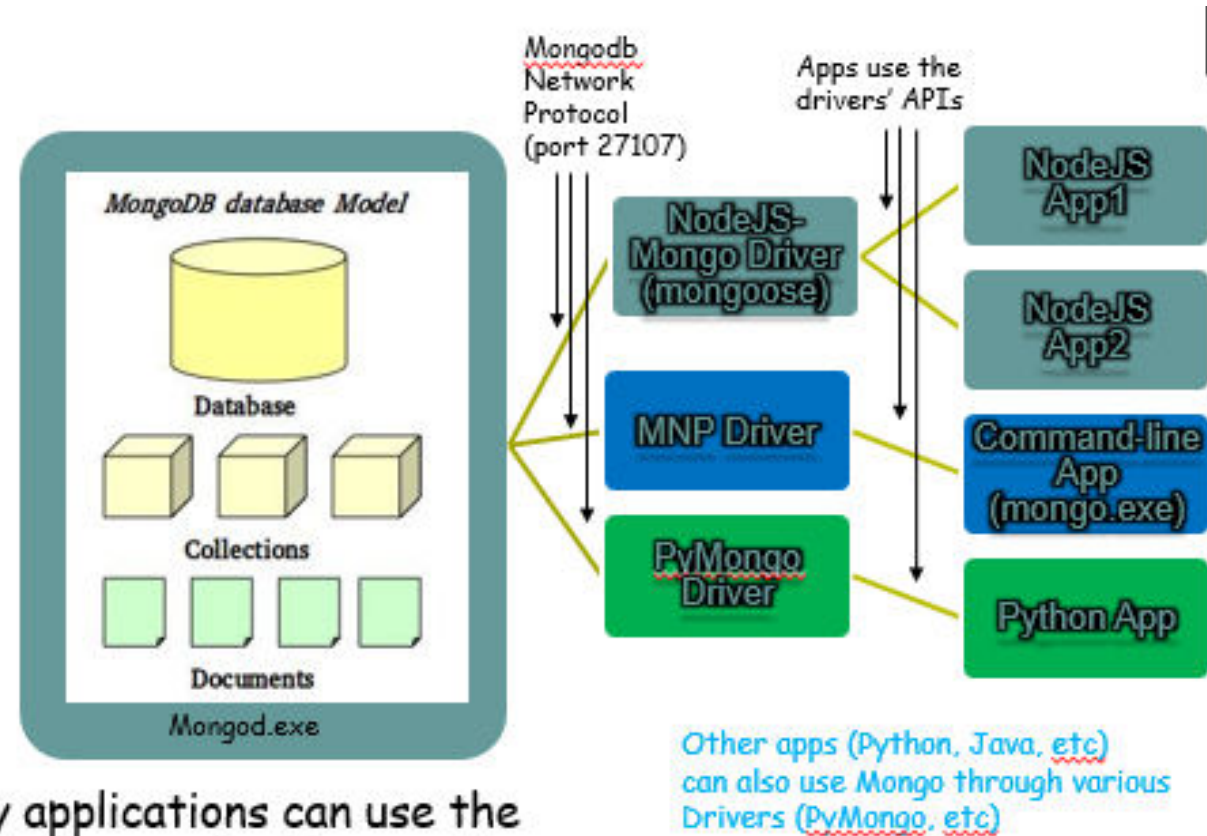


A MongoDB server may host many individual **databases** (A,B,C)

Each database may maintain many **collections**

Each collection may contain many **documents** (i.e. items)

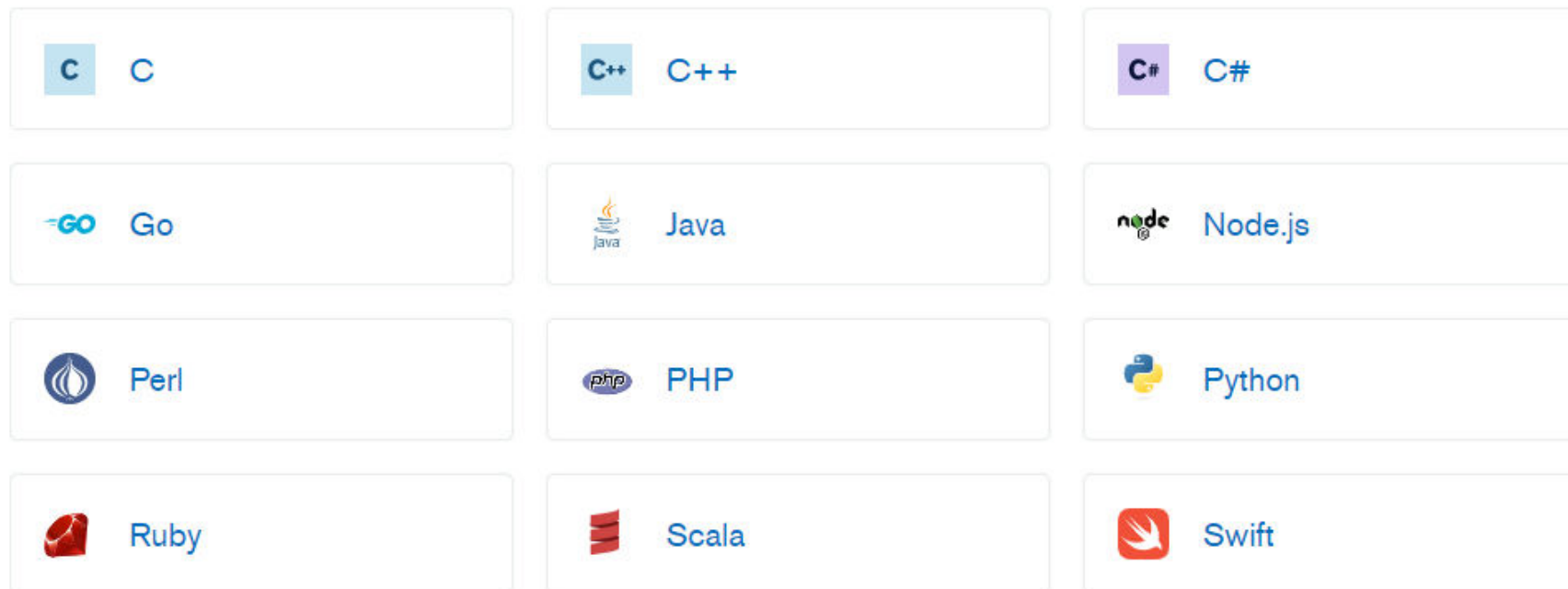
Interacting with MongoServer



Many applications can use the Mongo server **simultaneously** with no danger of overwriting data

Support for Other Programming Languages

- Drivers for various server-side programming language – <https://docs.mongodb.org/ecosystem/drivers/>



Mongo -Terminologies

Database:

- A container of MongoDB **collections**

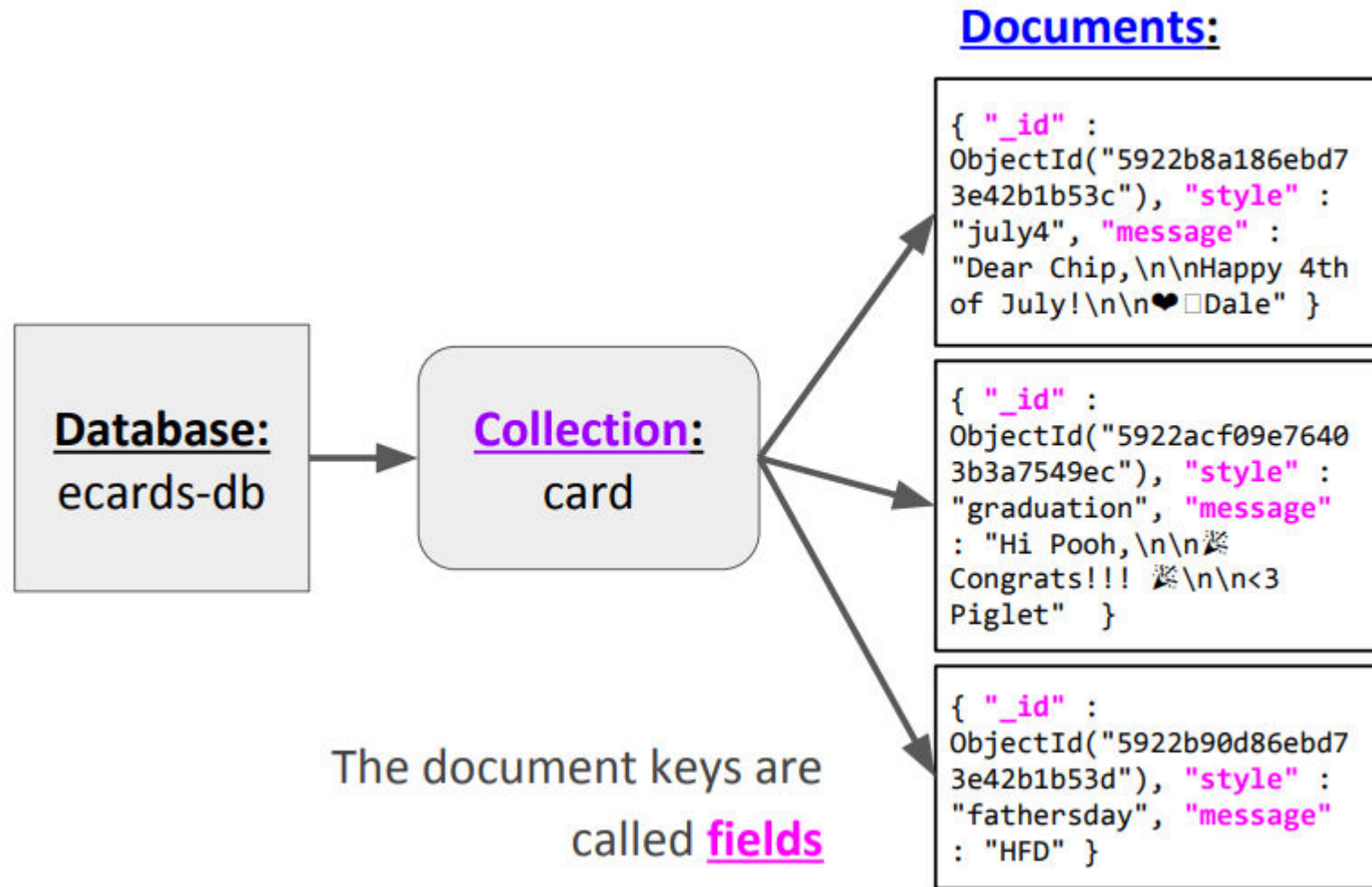
Collection:

- A group of MongoDB **documents**.
- (**Table** in a relational database)

Document:

- A JSON-like object that represents one instance of a collection (**Row** in a relational database)
- Also used more generally to refer to any set of key-value pairs.

Mongodb- Structure



Relational Vs MongoDB

- All the relational design knowledge still applies
 - Entities and relationships
 - Tables → collections

MySQL	MongoDB
ACID Transactions	ACID Transactions
Table	Collection
Row	Document
Column	Field
Secondary Index	Secondary Index
JOINS	Embedded documents, \$lookup & \$graphLookup
GROUP_BY	Aggregation Pipeline

- With that said, document databases are not relational databases
 - Doing JOIN is difficult
 - Does have much richer data types

A MongoDB Document ...

```
_id: ObjectId("5e92df51a3d02a5e20a0a539")
Name: "My First Blog Post"
Text: "First Blog"
Publish_Date: 2020-03-31T15:00:00.000+00:00
✓ Author: Object
  type: ObjectId("5e92d8f198ccaa271ca8f434")
✓ Comments: Object
  Comment: "Nice"
  Date: "2020-04-10 15:00:00:00"
  ✓ author: Object
    id: ObjectId("5e92d8f198ccaa271ca8f435")
    firstName: "Kashif"
    lastName: "Inam"
✓ Tag: Array
  0: "web development"
  1: "mongodb"
✓ Category: Array
  0: "CS"
  1: "Web"
```

A MongoDB Document

- Each document must have a unique `_id` field that serves as the primary key
- The value of `_id` can be user-assigned (of any type) or auto-generated (of [ObjectId](#) type)
- An `Object` value is also known as an *embedded document* or a *sub-document*

ObjectId

ObjectIds are small, likely unique, fast to generate, and ordered. ObjectId values are 12 bytes in length, consisting of:

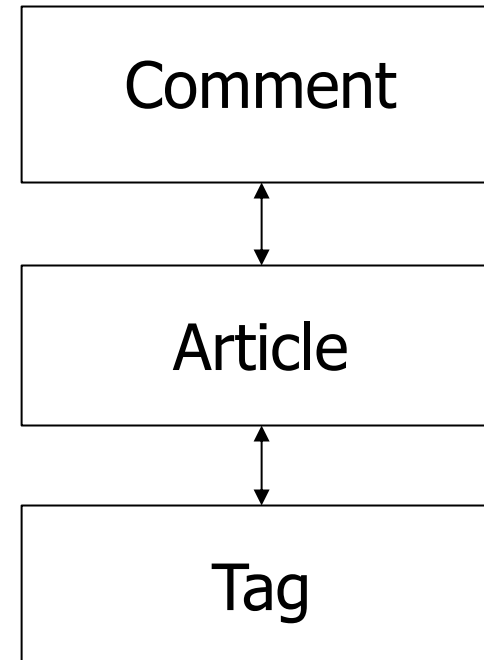
- a 4-byte *timestamp value*, representing the ObjectId's creation, measured in seconds since the Unix epoch
- a 5-byte *random value*
- a 3-byte *incrementing counter*, initialized to a random value

Embedded Document

- Embedded document model is a powerful concept, allowing you to **store all kinds of related information in a single document**.
- To embed or not to embed: that's the question

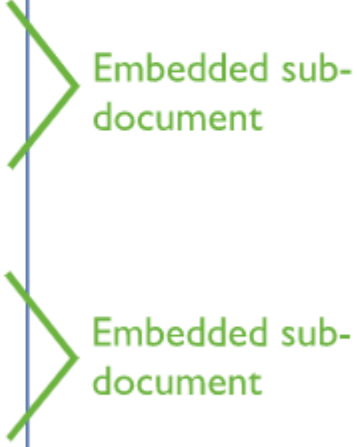


Or



Embedded Document

```
{
  _id: <ObjectId>,
  username: "123xyz",
  contact: {
    phone: "123-456-7890",
    email: "xyz@example.com"
  },
  access: {
    level: 5,
    group: "dev"
  }
}
```



Embedded sub-document

Embedded sub-document

Transaction and ACID

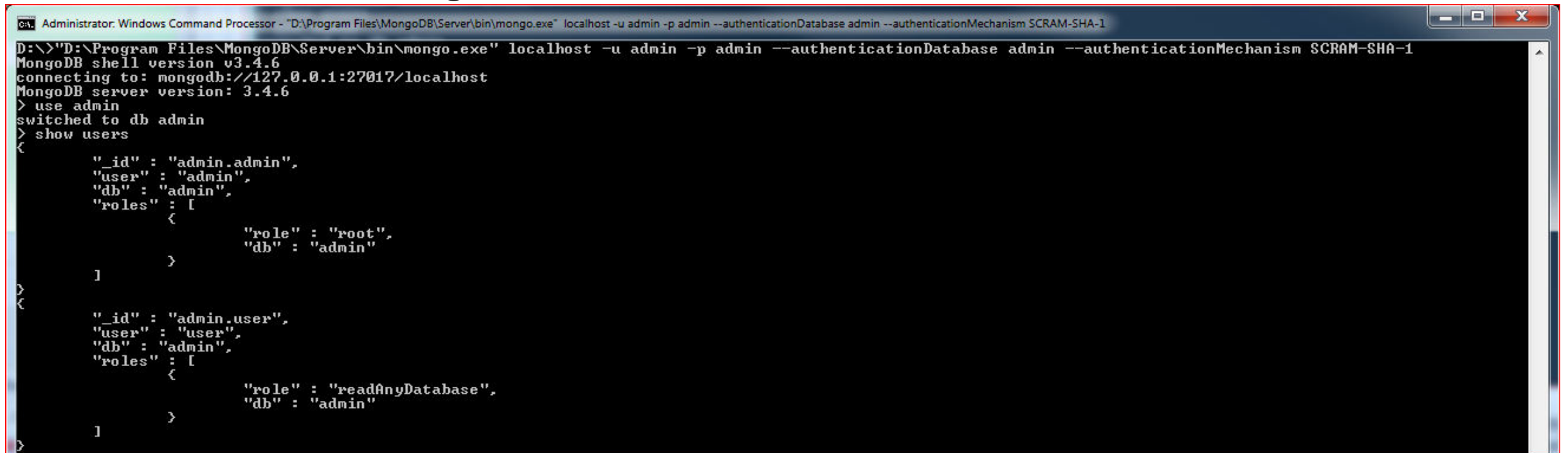
- A transaction in RDBMS is a group of SQL statements treated by the DBMS as a *single unit of work*
- Transactions in RDBMS are ACID
 - Atomic
 - Consistent
 - Isolated
 - Durable

Transactions in MongoDB

- Write operations are atomic on the level of a single document
- Multi-documents transaction support
 - Since 4.0 (6/2018) - MongoDB supports multi-document transactions on replica sets.
 - Since 4.2 (8/2019)
 - MongoDB introduces distributed transactions,
 - which adds support for multi-document transactions on shared clusters
 - and incorporates the existing support for multi-document transactions on replica sets.

MongoDB Shell

- mongo
- A command line client that provides an interactive JavaScript interface to MongoDB



```
Administrator: Windows Command Processor - "D:\Program Files\MongoDB\Server\bin\mongo.exe" localhost -u admin -p admin --authenticationDatabase admin --authenticationMechanism SCRAM-SHA-1
D:\>"D:\Program Files\MongoDB\Server\bin\mongo.exe" localhost -u admin -p admin --authenticationDatabase admin --authenticationMechanism SCRAM-SHA-1
MongoDB shell version v3.4.6
connecting to: mongodb://127.0.0.1:27017/localhost
MongoDB server version: 3.4.6
> use admin
switched to db admin
> show users
{
  "_id" : "admin.admin",
  "user" : "admin",
  "db" : "admin",
  "roles" : [
    {
      "role" : "root",
      "db" : "admin"
    }
  ]
}
{
  "_id" : "admin.user",
  "user" : "user",
  "db" : "admin",
  "roles" : [
    {
      "role" : "readAnyDatabase",
      "db" : "admin"
    }
  ]
}
```

Data Types

- Numbers
 - Boolean, Integer, Double, Decimal
- Text and binary data
 - String, Regular Expression, Code, Binary Data
- Date and timestamp (for internal use)
- Special types
 - Null, Min/Max Key, ObjectId
- Object and Array

Basic MongoDB Shell Commands

- `help`
- `show dbs`
- `use <db>`
 - **Switch to database <db>**
 - `show collections`
 - `db.dropDatabase()`

MongoDB's Query Language

- JavaScript with [MongoDB methods](#)

- Some collection methods:

- `db.<collection>.insert()`
- `db.<collection>.find()`
- `db.<collection>.update()`
- `db.<collection>.remove()`
- `db.<collection>.drop()`

```
await db.collection('inventory').updateOne(
  { item: 'paper' },
  {
    $set: { 'size.uom': 'cm', status: 'P' },
    $currentDate: { lastModified: true }
  }
);
```

```
SELECT * FROM inventory WHERE status = "A" AND qty < 30
const cursor = db.collection('inventory').find({
  status: 'A',
  qty: { $lt: 30 }
});
```

```
await db.collection('inventory').insertOne({
  item: 'canvas',
  qty: 100,
  tags: ['cotton'],
  size: { h: 28, w: 35.5, uom: 'cm' }
});
```

Comparison Operator

Comparison operators

- **\$eq, \$ne**
 - Tests the actual field value for **equality / inequality**
 - The same behavior as in case of value equality conditions
- **\$lt, \$lte, \$gte, \$gt**
 - Tests whether the actual field value is **less than / less than or equal / greater than or equal / greater than** the provided value
- **\$in**
 - Tests whether the actual field value is equal to **at least one** of the provided values
- **\$nin**
 - Negation of \$in

\$eq

\$gt

\$gte

\$in

\$lt

\$lte

\$ne

\$nin

\$and

\$not

\$nor

\$or

Operators

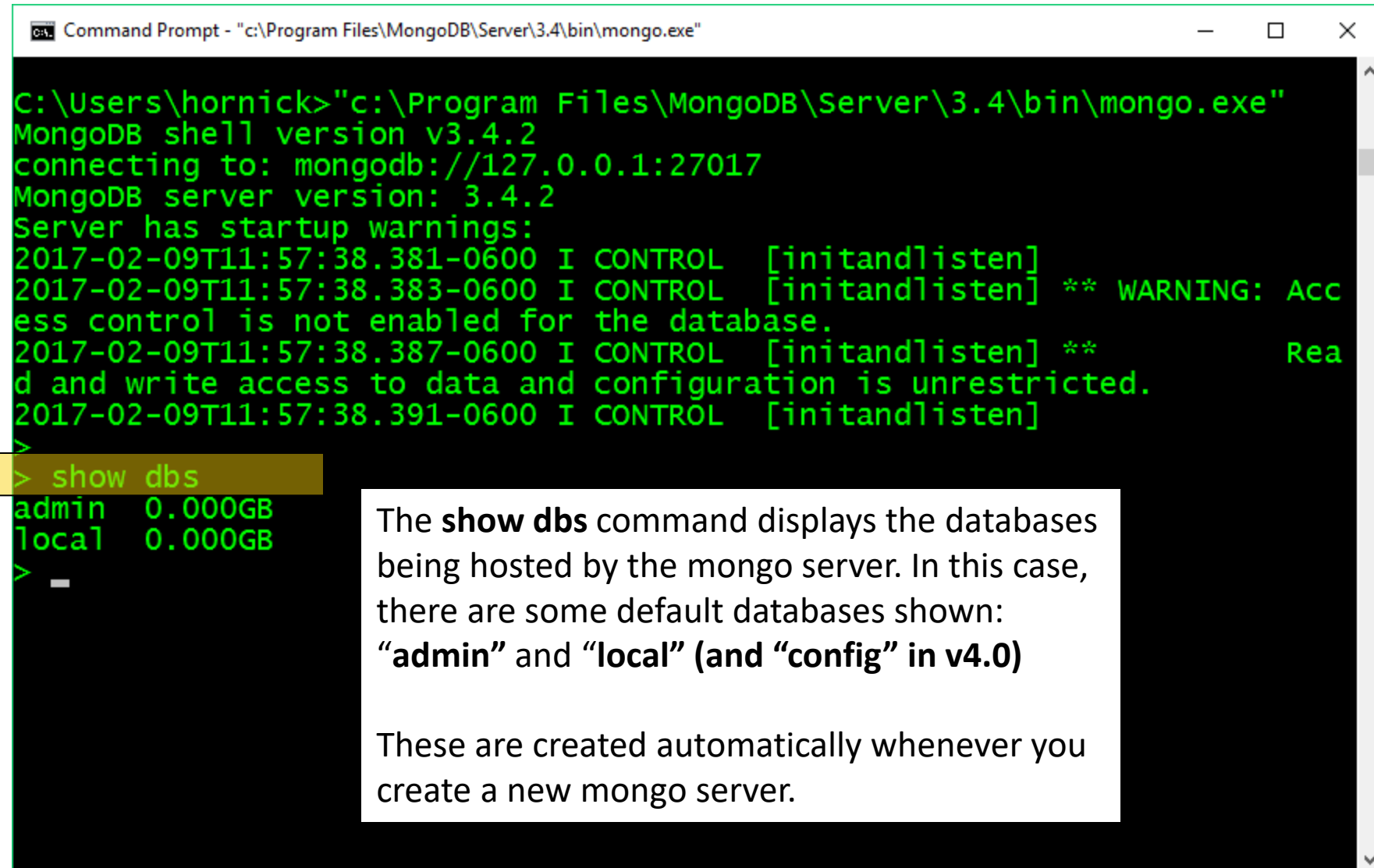
Name	Description
\$eq	Matches value that are equal to a specified value
\$gt, \$gte	Matches values that are greater than (or equal to a specified value
\$lt, \$lte	Matches values less than or (equal to) a specified value
\$ne	Matches values that are not equal to a specified value
\$in	Matches any of the values specified in an array
\$nin	Matches none of the values specified in an array
\$or	Joins query clauses with a logical OR returns all
\$and	Join query clauses with a logical AND
\$not	Inverts the effect of a query expression
\$nor	Join query clauses with a logical NOR
\$exists	Matches documents that have a specified field

Starting the MongoDB server (mongod.exe)

```
Command Prompt - "C:\Program Files\MongoDB\Server\3.4\bin\mongod.exe" --dbpath mongodata
C:\Users\hornick>"C:\Program Files\MongoDB\Server\3.4\bin\mongod.exe" --dbpath mongodata
2017-02-09T12:36:02.497-0600 I CONTROL [initandlisten] MongoDB starting : pid=3600 port=27017 dbpath=mo
ngodata 64-bit host=MSOE-5CG6292DR2
2017-02-09T12:36:02.501-0600 I CONTROL [initandlisten] targetMinOS: windows 7/Windows Server 2008 R2
2017-02-09T12:36:02.504-0600 I CONTROL [initandlisten] db version v3.4.2
2017-02-09T12:36:02.505-0600 I CONTROL [initandlisten] git version: 3f76e40c105fc223b3e5aac3e20dcd026b8
3b38b
2017-02-09T12:36:02.508-0600 I CONTROL [initandlisten] OpenSSL version: OpenSSL 1.0.1u-fips 22 Sep 201
6
2017-02-09T12:36:02.511-0600 I CONTROL [initandlisten] allocator: tcmalloc
2017-02-09T12:36:02.513-0600 I CONTROL [initandlisten] modules: none
2017-02-09T12:36:02.515-0600 I CONTROL [initandlisten] build environment:
2017-02-09T12:36:02.516-0600 I CONTROL [initandlisten]     distmod: 2008plus-ssl
2017-02-09T12:36:02.518-0600 I CONTROL [initandlisten]     distarch: x86_64
2017-02-09T12:36:02.520-0600 I CONTROL [initandlisten]     target_arch: x86_64
2017-02-09T12:36:02.522-0600 I CONTROL [initandlisten]     { "_id": "mongodata" } }
2017-02-09T12:36:02.525-0600 I CONTROL [initandlisten]     data created by the
'wiredTiger' storage engine
2017-02-09T12:36:02.530-0600 I CONTROL [initandlisten]     cache_size=3524M,
session_max=20000,eviction=
=true,path=journal,compressor=snappy),file_manager=(close_idle_time=100000),checkpoint=(wait=60,log_size
=2GB),statistics_log=(wait=0),
2017-02-09T12:36:03.054-0600 I CONTROL [initandlisten]
2017-02-09T12:36:03.056-0600 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for th
e database.
2017-02-09T12:36:03.059-0600 I CONTROL [initandlisten] **      Read and write access to data and co
nfiguration is unrestricted.
2017-02-09T12:36:03.061-0600 I CONTROL [initandlisten]
2017-02-09T12:36:03.156-0600 I FTDC [initandlisten] Initializing full-time diagnostic data capture w
ith directory 'mongodata/diagnostic.data'
```

Starting the server results in a lot of informational messages being displayed to the console.

Command-line viewer (mongo.exe)



```
Command Prompt - "c:\Program Files\MongoDB\Server\3.4\bin\mongo.exe"

C:\Users\hornick>"c:\Program Files\MongoDB\Server\3.4\bin\mongo.exe"
MongoDB shell version v3.4.2
connecting to: mongodb://127.0.0.1:27017
MongoDB server version: 3.4.2
Server has startup warnings:
2017-02-09T11:57:38.381-0600 I CONTROL [initandlisten]
2017-02-09T11:57:38.383-0600 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for the database.
2017-02-09T11:57:38.387-0600 I CONTROL [initandlisten] ** Read and write access to data and configuration is unrestricted.
2017-02-09T11:57:38.391-0600 I CONTROL [initandlisten]
>
> show dbs
admin 0.000GB
local 0.000GB
>
```

The **show dbs** command displays the databases being hosted by the mongo server. In this case, there are some default databases shown: **"admin"** and **"local"** (and **"config"** in v4.0)

These are created automatically whenever you create a new mongo server.

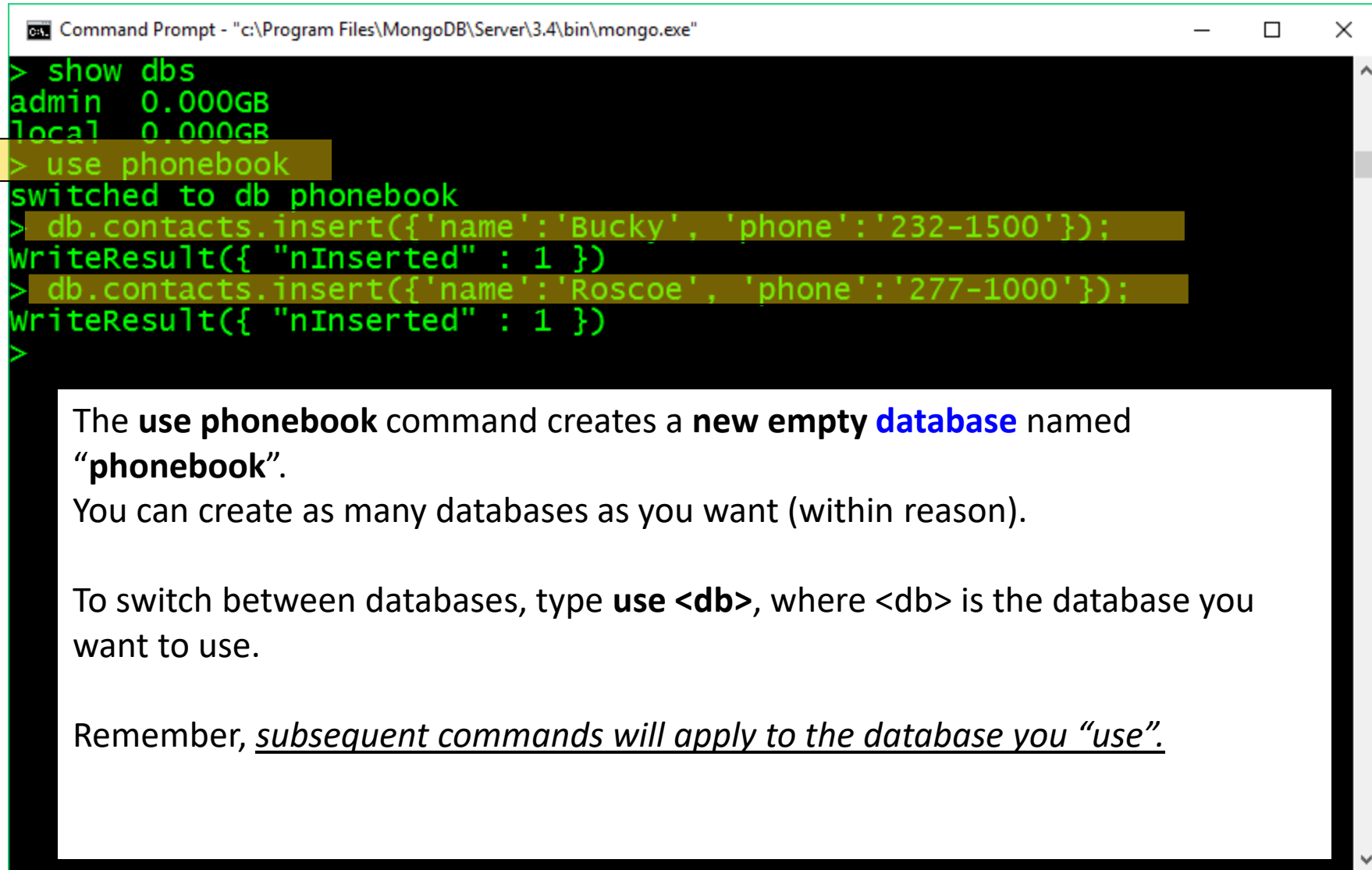
Creating the Collection in MongoDB on the fly

- The cool thing about MongoDB is that you need not to create collection before you insert document in it. With a single command you can insert a document in the collection and the MongoDB creates that collection on the fly.

Syntax: **db.collection_name.insert({key:value, key:value...})**

- Use MyTestDb-
- `db.Users.insert({ name:"Maria",age:30, Website:"linkedin.com"})`
- `db.Users.find()`
- show collections

Creating a new database



```
Command Prompt - "c:\Program Files\MongoDB\Server\3.4\bin\mongo.exe"
> show dbs
admin 0.000GB
local 0.000GB
> use phonebook
switched to db phonebook
> db.contacts.insert({'name': 'Bucky', 'phone': '232-1500'});
WriteResult({"nInserted" : 1 })
> db.contacts.insert({'name': 'Roscoe', 'phone': '277-1000'});
WriteResult({"nInserted" : 1 })
>
```

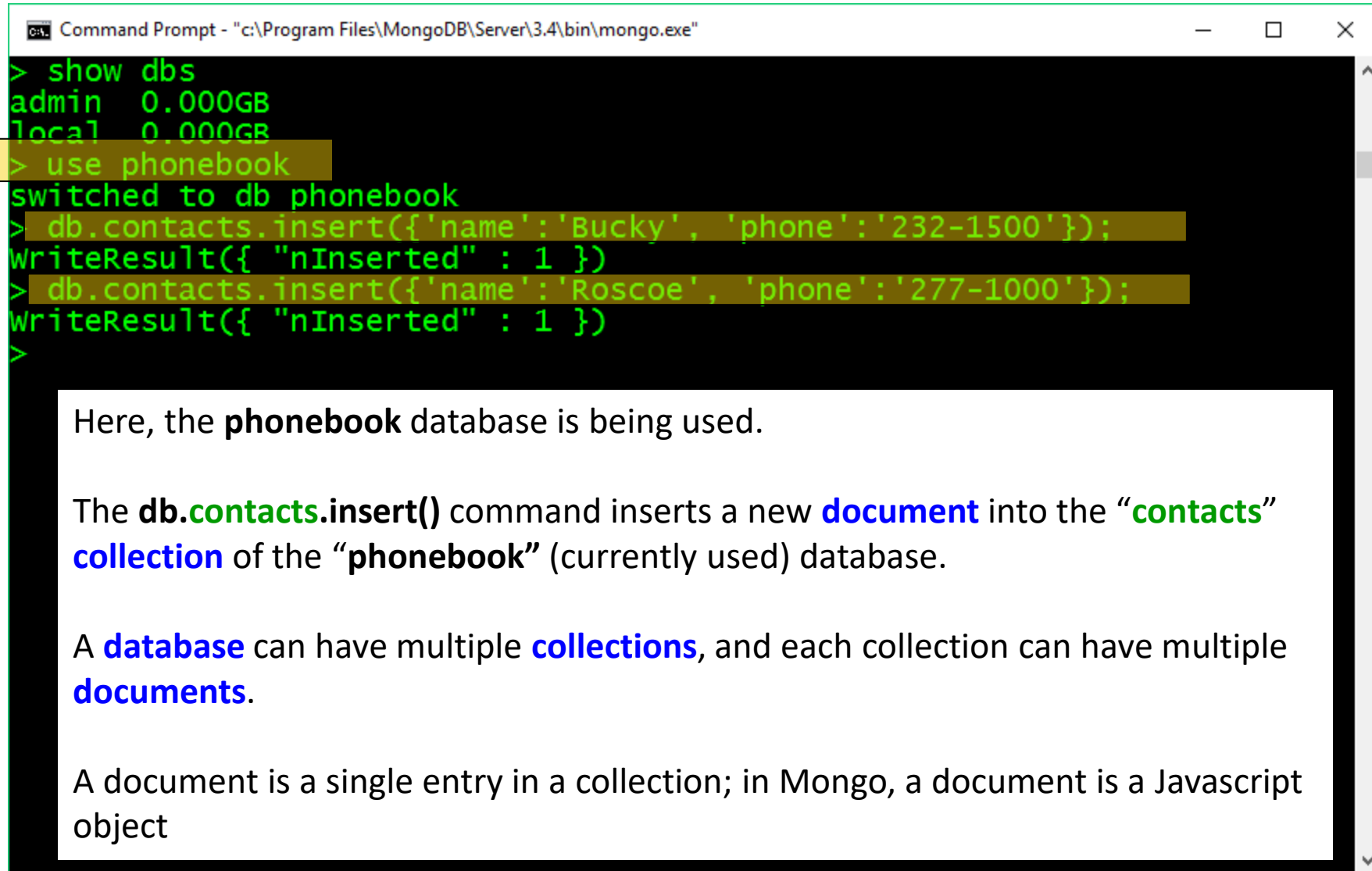
The **use phonebook** command creates a **new empty database** named “**phonebook**”.

You can create as many databases as you want (within reason).

To switch between databases, type **use <db>**, where <db> is the database you want to use.

Remember, subsequent commands will apply to the database you “use”.

Creating/inserting documents into a db collection

A screenshot of a Windows Command Prompt window titled "Command Prompt - \"c:\\Program Files\\MongoDB\\Server\\3.4\\bin\\mongo.exe\"". The window has a black background with green text. The commands and output are as follows:

```
> show dbs
admin 0.000GB
local 0.000GB
> use phonebook
switched to db phonebook
> db.contacts.insert({'name': 'Bucky', 'phone': '232-1500'});
WriteResult({ "nInserted" : 1 })
> db.contacts.insert({'name': 'Roscoe', 'phone': '277-1000'});
WriteResult({ "nInserted" : 1 })
>
```

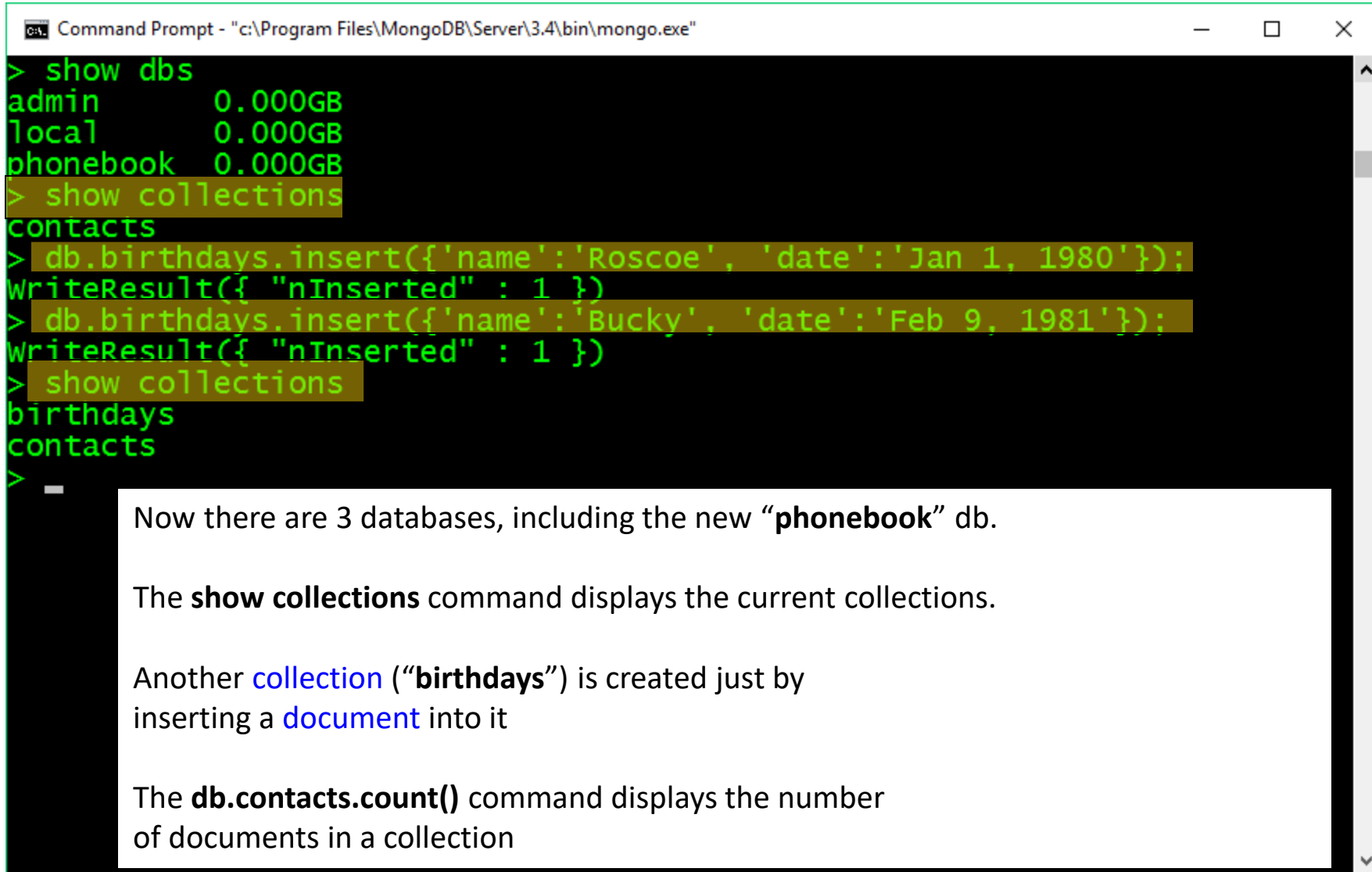
Here, the **phonebook** database is being used.

The **db.contacts.insert()** command inserts a new **document** into the “**contacts**” **collection** of the “**phonebook**” (currently used) database.

A **database** can have multiple **collections**, and each collection can have multiple **documents**.

A document is a single entry in a collection; in Mongo, a document is a Javascript object

Viewing databases and collections



```
Command Prompt - "c:\Program Files\MongoDB\Server\3.4\bin\mongo.exe"
> show dbs
admin      0.000GB
local      0.000GB
phonebook  0.000GB
> show collections
contacts
> db.birthdays.insert({'name':'Roscoe', 'date':'Jan 1, 1980'});
WriteResult({ "nInserted" : 1 })
> db.birthdays.insert({'name':'Bucky', 'date':'Feb 9, 1981'});
WriteResult({ "nInserted" : 1 })
> show collections
birthdays
contacts
>
```

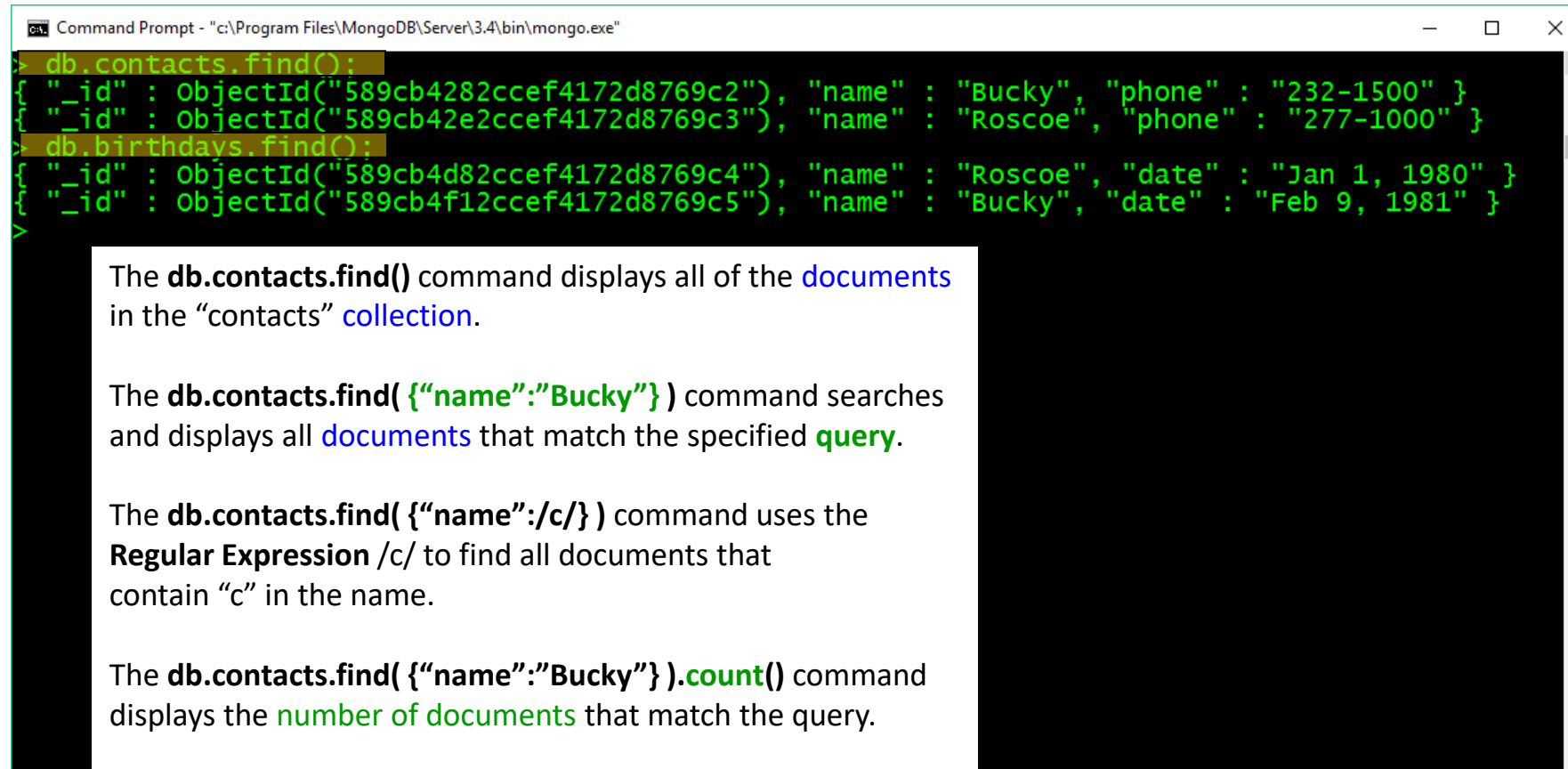
Now there are 3 databases, including the new “**phonebook**” db.

The **show collections** command displays the current collections.

Another **collection** (“**birthdays**”) is created just by inserting a **document** into it

The **db.contacts.count()** command displays the number of documents in a collection

Searching and Viewing documents/records in a collection



```
Command Prompt - "c:\Program Files\MongoDB\Server\3.4\bin\mongo.exe"
> db.contacts.find():
{"_id" : ObjectId("589cb4282ccef4172d8769c2"), "name" : "Bucky", "phone" : "232-1500" }
{"_id" : ObjectId("589cb42e2ccef4172d8769c3"), "name" : "Roscoe", "phone" : "277-1000" }
> db.birthdays.find():
{"_id" : ObjectId("589cb4d82ccef4172d8769c4"), "name" : "Roscoe", "date" : "Jan 1, 1980" }
{"_id" : ObjectId("589cb4f12ccef4172d8769c5"), "name" : "Bucky", "date" : "Feb 9, 1981" }
```

The **db.contacts.find()** command displays all of the **documents** in the “contacts” **collection**.

The **db.contacts.find({“name”:”Bucky”})** command searches and displays all **documents** that match the specified **query**.

The **db.contacts.find({“name”:/c/})** command uses the **Regular Expression** /c/ to find all documents that contain “c” in the name.

The **db.contacts.find({“name”:”Bucky”}).count()** command displays the **number of documents** that match the query.

Each **document** is a Javascript representation of the data that was inserted – but note that Mongo prefixes each document with a unique id.

Queries : Basic Conditions and Projections

- List all users
- List the first name of all users
 - Without _id
- Find the users whose last name is Jilani
 - Using [comparison operators](#)

```
const cursor = db.collection('User').find({});
```

```
const cursor = db.collection('User').find({}).project({ Name: 1, _id: 0, });
```

```
const cursor = db.collection('User').find({Name: ' John Doe '});
```

Deleting documents/records in a collection

```
Command Prompt - "C:\Program Files\MongoDB\Server\3.6\bin\mongo.exe"
> db.contacts.insert( { 'name': 'Roscoe', 'phone': '274-2000' } );
WriteResult({ "nInserted" : 1 })
> db.contacts.insert( { 'name': 'Roscoe', 'phone': '276-3400' } );
WriteResult({ "nInserted" : 1 })
> db.contacts.find();
{ "_id" : ObjectId("5a81f4a5289d89cdd456c658"), "name" : "Roscoe", "phone" : "277-1000" }
{ "_id" : ObjectId("5a81f7fc289d89cdd456c65f"), "name" : "Bucky", "phone" : "232-1500" }
{ "_id" : ObjectId("5a81f840289d89cdd456c660"), "name" : "Roscoe", "phone" : "274-2000" }
{ "_id" : ObjectId("5a81f84c289d89cdd456c661"), "name" : "Roscoe", "phone" : "276-3400" }
> db.contacts.deleteOne( { "name": "Roscoe" } );
{ "acknowledged" : true, "deletedCount" : 1 }
> db.contacts.find();
{ "_id" : ObjectId("5a81f7fc289d89cdd456c65f"), "name" : "Bucky", "phone" : "232-1500" }
{ "_id" : ObjectId("5a81f840289d89cdd456c660"), "name" : "Roscoe", "phone" : "274-2000" }
{ "_id" : ObjectId("5a81f84c289d89cdd456c661"), "name" : "Roscoe", "phone" : "276-3400" }
> db.contacts.deleteMany( { "name": "Roscoe" } );
{ "acknowledged" : true, "deletedCount" : 2 }
> db.contacts.find();
{ "_id" : ObjectId("5a81f7fc289d89cdd456c65f"), "name" : "Bucky", "phone" : "232-1500" }
```

The **db.contacts.deleteOne(...)** command deletes the first **document** matching the specified **query** from the “**contacts**” **collection**.

The **db.contacts.deleteMany(...)** command deletes **all** **documents** matching the specified **query** from the “**contacts**” **collection**.

Deleting collections and entire databases

```
Command Prompt - "C:\Program Files\MongoDB\Server\3.6\bin\mongo.exe"
> db.contacts.find({"name": "Bucky"});
{ "_id" : ObjectId("5a81f7fc289d89cdd456c65f"), "name" : "Bucky", "phone" : "232-1500" }
> show collections
birthdays
contacts
> db.birthdays.find();
{ "_id" : ObjectId("5a81f518289d89cdd456c65b"), "name" : "Bucky", "date" : "Feb 9, 1981" }
{ "_id" : ObjectId("5a81f528289d89cdd456c65c"), "name" : "Roscoe", "date" : "Jan 1, 1980" }
> db.birthdays.drop();
true
> show collections
contacts
> db.dropDatabase();
{ "dropped" : "phonebook", "ok" : 1 }
> show dbs
admin    0.000GB
config  0.000GB
local    0.000GB
> _
```

The **db.birthdays.drop()** command deletes the “**birthdays**” collection from the current database.

The **db.dropDatabase()** command deletes the current database itself from the server

Example of Basic Operations

- Create a database `test1`
- Create two documents `Atif` and `Amir`
- Save the two documents to a collection `Users`
- List the documents in the collection

```
await db.collection('User').insertMany([
  { firstName: 'Atif', firstName: 'Jilani', email: 'atif.jilani@nu.edu.pk' },
  { firstName: 'Amir', firstName: 'Waqar', email: 'waqar.amir@gmail.com' }
]);
```

```
1  'use strict';
2
3  const mongoose = require('mongoose');
4
5  let userSchema = new mongoose.Schema({
6    firstName: String,
7    lastName: String,
8    email: {
9      type: String,
10     required: true,
11     unique: true,
12     lowercase: true,
13     trim: true
14   }
15 });
16
17 userSchema.methods.name = function () {
18   return this.firstName + ' ' + this.lastName;
19 }
20
21 userSchema.statics.findByEmail = function (email, callback) {
22   return this.findOne({
23     email: email
24   }, callback);
25 }
26
27 module.exports = mongoose.model('User', userSchema);
```

Using find()

- `find( [query], [projection] )`
 - [Query tutorials](#)
 - [Query operators](#)
- Examples using the Blogs database

```
SELECT * FROM User WHERE firstName = "Atif " AND email="atif.jilani@nu.edu.pk")
```

```
const cursor = db.collection('User').find({  
  $and: [{Name: 'Atif' }, { email: 'atif.jilani@nu.edu.pk'}] });
```

```
const cursor = db.collection('User').find({}).project({ firstName: 1, _id: 0, });
```

Queries - Arrays and Sub-documents

- Find the articles whose tags contain "NoSQL"
 - Using `$all`
- Find the articles commented on 04-12-2020

```
db.articles.find( { Tag: { $all:["NoSQL"] } } )
```

```
db.collection('Article').find( {  
  Comments: { $all: [ { "$elemMatch" : { Comment:"Intresting", Date:"04-12-2020" }  
                    }  
                  ]  
                }  
} )
```

Example

- To find all products released last year (2015) or later costing under \$100
- `db.products.find( {released: {$gte: new Date(2015, 1, 1)}, price {$lte': 100}},)`

Exercise with example

-
- **All books**
`db.bib.find()`
 - **Books with title “Foundations of Databases”**
`db.bib.find( { title: "Foundations of Databases" } )`
 - **Books whose title contains “Database” or “database” and whose price is lower than \$50**
`db.bib.find( { title: /[dD]atabase/, price: { $lt: 50 } } )`
 - **Books with price between \$70 and \$100**
`db.bib.find( { $and: [ { price: { $gte: 70 } }, { price: { $lte: 100 } } ] } )`
 - By the way, why wouldn't the following work?
`db.bib.find( { price: { $gte: 70 }, price: { $lte: 100 } } )`
 - **Books authored by Widom**
`db.bib.find( { authors: "Widom" } )`
- Assume db refers to the database and db.bib refers to the collection of books
 - Add .toArray() at end to get pretty output
 - You need to do this for Homework 3!

Queries (IV) Join

- List the articles with their authors (i.e. not just author id)
 - `db.<collection>.aggregate()`
 - [`\$lookup`](#)
- List the article authors
 - [`\$project`](#)

```
const docs = await db.collection('Article').aggregate([
  {
    $lookup: {
      from: 'User',
      localField: 'Author',
      foreignField: '_id',
      as: 'user'
    }
  }
],
{
  $unwind: '$user'
},
{
  $project: {
    _id: 0,
    Author_Name: '$user.Name',
    Author_Email: '$user.email',
    Title: 1,
    Tag: 1,
    publishDate: 1
  }
});
```

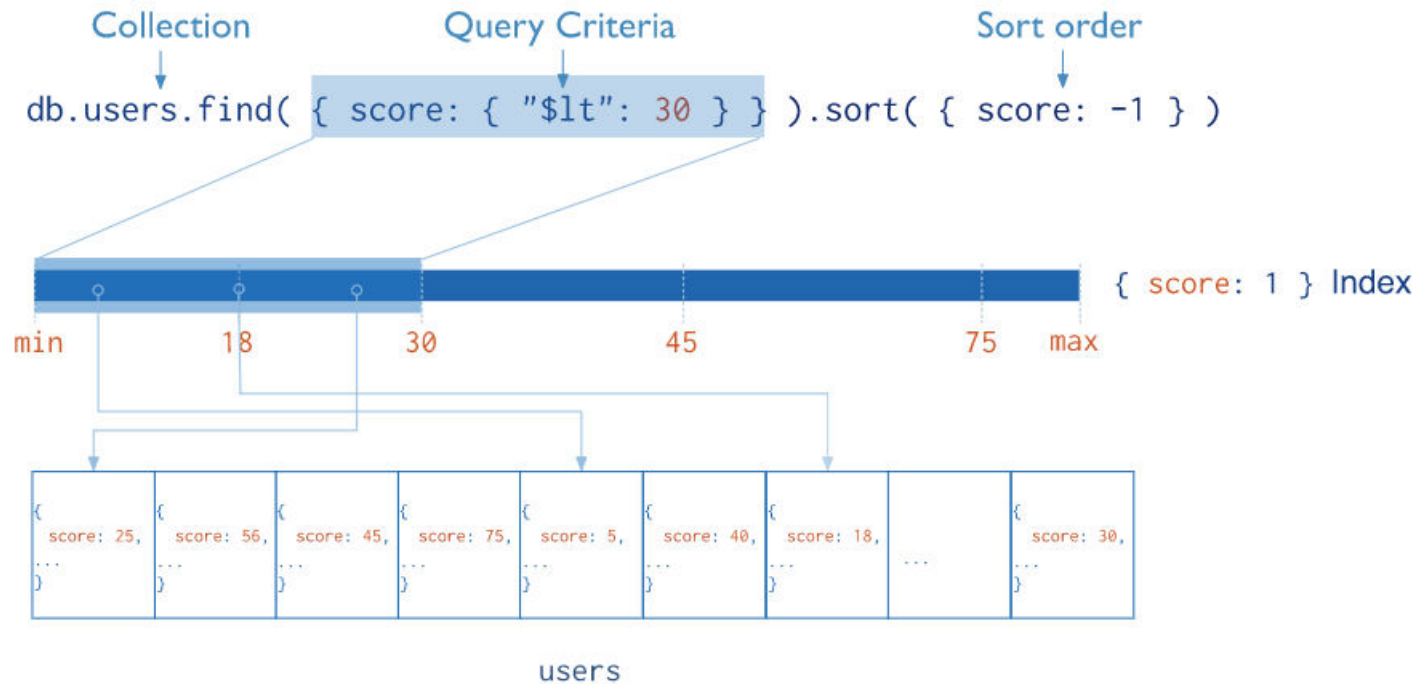
Update and Delete

[db.<collection>.update\(query, update, options \)](#)
[db.<collection>.remove\(query, update, options \)](#)

- Change Atif Jilani's name to Aftab Ahmed
 - [\\$set](#)
 - Other [update operators](#)
- Delete the article "Using MongoDB"
- Add a tag "Tutorial" to the article "Programming Node.js"
 - [\\$push](#)
- Delete the comments made by John Doe
 - [\\$pull](#)
 - `db.inventory.update({"item": "paper"},{$set: {"qty": 101}});`
 - `db.student.remove({"regNo":"3014"})`

Indexing

- Indexes are crucial for performance just like in RDBMS
- `db.<collection>.createIndex(<keys>, [options])`
 - keys: {field: -1 | 1, field2: -1 | 1, ...}
 - options: unique, name, ...



Index Examples

Ascending order



```
db.users.createIndex({lastName: 1});
```

```
db.users.createIndex({email: 1}, {  
  unique: true,  
  name: 'EmailIndex'  
});
```

```
db.articles.createIndex({tags: 1});
```



Index on array field

MongoDB with Node.js

- MongoDB Node.js driver
- [Mongoose](#)
 - Model classes
 - Validation
 - DAO methods

```
var express = require('express');
var app = express();
const userRoute = express.Router();
const MongoClient = require('mongodb').MongoClient;
const assert = require('assert');

// Connection URL
const url = 'mongodb://localhost:27017';
// Database Name
const dbName = 'myproject';
// Create a new MongoClient
const client = new MongoClient(url);

app.get('/', function (req, res) {

  // Use connect method to connect to the Server
  client.connect(function(err) {
    assert.equal(null, err);
    const db = client.db(dbName);
    console.log("Connected successfully to DB server");

    insertUser(db, function() {
      findUser(db, function() {
        client.close();
      });
    });
  });
});
```

Sample Data

Collection of **movies**

```
{
  _id: ObjectId("1"),
  title: "Vratné lahve", year: 2006,
  actors: [ ObjectId("7"), ObjectId("5") ]
}
```

```
{
  _id: ObjectId("2"),
  title: "Samotáři", year: 2000,
  actors: [ ObjectId("6"), ObjectId("4"),
            ObjectId("5") ]
}
```

```
{
  _id: ObjectId("3"),
  title: "Medvídek", year: 2007,
  actors: [ ObjectId("5"), ObjectId("4") ]
}
```

Collection of **actors**

```
{ _id: ObjectId("4"),
  firstname: "Ivan",
  lastname: "Trojan" }
```

```
{ _id: ObjectId("5"),
  firstname: "Jiří",
  lastname: "Macháček" }
```

```
{ _id: ObjectId("6"),
  firstname: "Jitka",
  lastname: "Schneiderová" }
```

```
{ _id: ObjectId("7"),
  firstname: "Zdeněk",
  lastname: "Svěrák" }
```

Insert Operation :Example

Insert a new actor document

```
db.actors.insert(  
  {  
    firstname: "Anna",  
    lastname: "Geislerová"  
  }  
)
```

```
{  
  _id: ObjectId("8"),  
  firstname: "Anna",  
  lastname: "Geislerová"  
}
```

Insert two new movies

```
db.movies.insert(  
  [  
    {  
      _id: ObjectId("9"), title: "Želary", year: 2003,  
      actors: [ ObjectId("4"), ObjectId("8") ]  
    },  
    { title: "Anthropoid", year: 2016, actors: [ ObjectId("8") ] },  
  ]  
)
```

Find Operation: Example

Select all movies from our collection

```
db.movies.find()
```

```
db.movies.find( { } )
```

Select a particular movie based on its document identifier

```
db.movies.find( { _id: ObjectId("2") } )
```

Select movies filmed in *2000* with a rating greater than *1*

```
db.movies.find( { year: 2000, rating: { $gt: 1 } } )
```

Select movies filmed between *2005* and *2015*

```
db.movies.find( { year: { $gte: 2005, $lte: 2015 } } )
```

Projection -Example

Find a particular movie, select its identifier, title and actors

```
db.movies.find(  
  { _id: ObjectId("2") },  
  { title: true, actors: true }  
)
```

```
{  
  _id: ObjectId("2"),  
  title: "Samotáři",  
  actors: [ ObjectId("6"),  
            ObjectId("4"),  
            ObjectId("5") ]  
}
```

Find movies from 2000, select their titles and the last two actors

```
db.movies.find(  
  { year: 2000 },  
  {  
    title: 1, _id: 0,  
    actors: { $slice: -2 }  
  }  
)
```

```
{  
  title: "Samotáři",  
  actors: [ ObjectId("4"),  
            ObjectId("5") ]  
}
```

Update Operation: Example

Replace the whole document of at most one specified actor

```
db.actors.update(  
  { _id: ObjectId("8") },  
  { firstname: "Aña",  
    lastname: "Geislerová" }  
)
```

```
{  
  _id: ObjectId("8"),  
  firstname: "Aña",  
  lastname: "Geislerová"  
}
```

Update all movies filmed in 2015 or later

```
db.movies.update(  
  { year: { $gt: 2015 } },  
  {  
    $set: { new: true },  
    $inc: { rating: 3 }  
  },  
  { multi: true }  
)
```

Thank You